Design layer

the whole layer, as one document

Contents

1 JSON Blueprint 4

1.1 Foundation 4
1.2 System at a glance 5
1.3 Repository verification 5
1.3.1 Gate and continuous integration 5
1.3.2 Executable documentation 6
1.3.3 Measurement 6

2 JSON contracts 8

2.1 Foundation 8
2.2 Pending updates 10
2.3 System at a glance 11
2.4 Whole model 11
2.5 Wire values 14
2.5.1 Exact numbers 14
2.5.2 Strict document admission 15
2.5.3 Rendering and standard JSON bridges 15
2.6 Typed codecs 16
2.6.1 Constructor acceptance 16
2.6.2 Record construction and dependent fields 17
2.6.3 Tagged union construction 18
2.6.4 Mappings and custom conversion 18
2.6.5 Paths and safe diagnostics 19
2.7 Runtime contracts 19
2.7.1 Schema discovery and public projection 19
2.7.2 Document loading profile 19
2.7.3 Normalization and schema matching 20
2.7.4 Validation and native conversion 21
2.8 Legacy compatibility 21
2.9 Unsupported schema families 22
2.10 Verification and host boundaries 23
2.10.1 Oracle and behavioral evidence 23
2.10.2 Native host boundaries 23
2.11 End-to-end walkthrough 24

3 Source generation 26

3.1 Foundation 26
3.2 Pending updates 27
3.3 System at a glance 28
3.4 Source generation 28
3.4.1 Typed-definition generator 28
3.4.2 Schema materialization 29
3.4.3 Schema-driven native generator 29
3.4.4 Generated names and failures 30
3.4.5 Generated parity and verification 32

Glossary 33

ROOT DESIGN

JSON Blueprint

1 JSON Blueprint

1.1 Foundation

Two contexts divide the package: contracts owns runtime values, codecs, schemas and validation, and generation owns development-time source generation. Each context states its own goals, invariants and principles.

NO-GOAL

Execute or authorize application operations

Tool registration, provider admission, transport, job execution, workflow execution, and application policy belong to consumers. Ordinary JSON serialization can use standard Gleam JSON without this package.

NO-GOAL

Claim general JSON Schema conformance

The Finite profile has bounded independent evidence. A Draft 2020-12 dialect label does not imply support for every keyword or an arbitrary remote schema.

INVARIANT

Runtime conversion never invokes generation

The package includes both contexts. Runtime conversion never invokes source generation; the application build explicitly calls generation and compiles its returned source. The consumer check compiles a production copy without development dependencies.

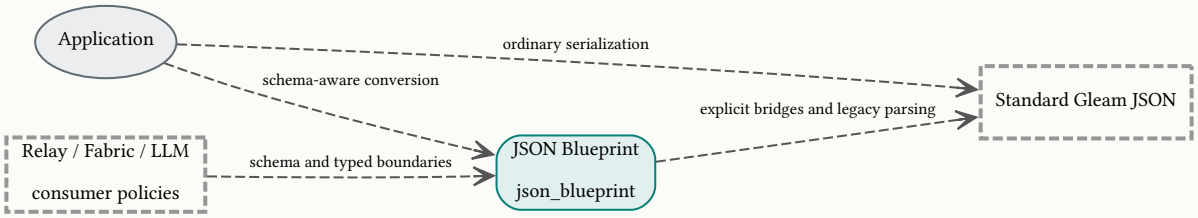
mechanism

- Ownership and finite support are recorded in ADR-0001. Generation placement is recorded in ADR-0008. The owner selected version 2.0.0 for the consolidated package in ADR-0013, superseding the hold in ADR-0011. The migration guide separates legacy API upgrade changes from adopting codec wire formats.

1.2 System at a glance

One package supplies pure values, schemas, codecs, contracts and explicit source generation.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Application-owned composition

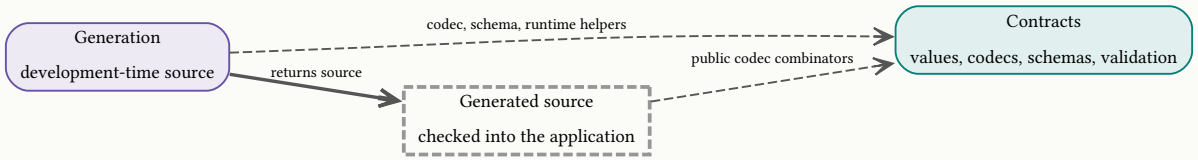


LEGEND



Arrows declare dependency direction. Consumers select their own protocol and provider admission policy.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L2 · CONTEXTS Contexts



LEGEND



The generation context depends on the contracts context and never the reverse. Generated source depends on the contracts runtime only.

- The package exports pure values and synchronous conversion operations. It starts no process, owns no connection, and stores no mutable runtime state.
- `contracts` holds Wire values, Typed codecs, Runtime contracts and Legacy compatibility. `generation` holds the typed-definition generator, schema materialization and the schema-driven native generator. The context map declares their relationship.
- Relay owns protocol declarations and revision-specific generation. Fabric owns tool and model execution. LLM adapters own provider lowering and admission. These consumers inspect opaque schema children; they never acquire authority from a dialect label alone.
- No operation supplies authorization, identifiers for business entities, correlation, idempotency keys, retries, cancellation, or a wall-clock deadline. Applications own those capabilities where their operation requires them.

1.3 Repository verification

Verification distinguishes finite acceptance, native projection, legacy behavior, generated parity, executable documentation and measurement.

1.3.1 Gate and continuous integration

Check	Scope
Formatting	A Nix check formats a copied source tree; frozen oracle bytes, generated fixtures, rendered design evidence and build output are excluded
Native compilation	Authored Erlang FFI compiles independently with warnings as errors
Package tests	Compilation with warnings as errors and the test suite on Erlang and JavaScript for the runtime and generators in the root package

Check	Scope
Usage examples	The examples package builds with warnings as errors, runs its tests on Erlang and JavaScript, and runs once; every usage topic and the legacy and migration forms are compiled code
Schema consumer	Fresh generation in disposable packages, tests on both targets, and an isolated production build without development dependencies
Independent oracles	Python checkers compare schemas with a Draft 2020-12 validator, exact arithmetic with Decimal/Fraction, and both-target parsing with a pinned upstream corpus and generated cases. The oracle document pins the references and states the limits of each comparison

- The package gate removes each package's build output first, so an obsolete compiled module cannot hide a missing source module. Runtime logs are evidence and never substitute for compiler diagnostics.
- Continuous integration runs the gate on Erlang/OTP and Node.js with formatting and compilation as the required checks. Nix pins the compiler, the runtimes and the Python oracle environment. Browser JavaScript is not tested.
- CI invokes the shared treefmt and design-layer validators directly. Local scripts own application behavior; no local formatter or design-check wrappers are maintained. ADR-0012 The package gate and the development environment define operational verification.

1.3.2 Executable documentation

- Usage is compiled code. Each topic is a module of the examples package, which the gate builds and runs on both targets. The shared pinned Snips tool synchronizes README snippets from marked regions of compiled, tested examples. Local bespoke snippet-equality checkers remain removed. The legacy 1.x forms and the move to a codec are compiled tests under examples/test. Public module examples and the codegen consumer remain executable behavioral tests.
- Prose and checked examples stay aligned. Documentation that states a limit or default names the design section that owns it.

1.3.3 Measurement

- Benchmarks are observations of a named checkout. No timing threshold enters a gate, and the ordinary gate keeps the semantic tests of the benchmark fixtures. ADR-0010 records the policy and the accepted findings.
- Each capture starts three fresh VMs per target, runs an untimed warmup, and reports the median of the VM medians with the range of those medians. A capture records its source fingerprints, lockfile hashes, toolchain versions and host. Comparisons run the baseline before the changed code in fixed order.
- Complete-text, Value-only and construction cases measure different work. A comparison matches cases with the same parser policy: generated native decoding uses the standard JSON parser and strict decoding uses the package parser, with different number, duplicate-key and resource guarantees.
- The weekly benchmark workflow runs the harness on both targets. The benchmark script writes a fresh evidence directory and refuses to reuse one. The benchmark document retains the method, environment and dated observations.

CONTEXT

JSON contracts

2 JSON contracts

2.1 Foundation

One definition supplies typed conversion and an inspectable JSON contract.

GOAL

Describe schema-aware data once

Applications retain one Codec for JSON conversion, schema publication, and schema-aware composition. Runtime schema documents produce contracts for wire validation.

GOAL

Reject hostile input within explicit bounds

Strict text admission retains exact numbers and duplicate-key evidence. A byte limit bounds admission; consumers own depth, value-count and numeric policies.

GOAL

Keep the published 1.x API available

The frozen 1.x API remains available for its established wire formats and recursive types.

INVARIANT

Number comparison preserves mathematical value

Exact number comparison and integrality never pass through Float. Native construction and projection check target representation limits.

mechanism

INVARIANT

A witness retains the value that was validated

A Validated value retains an immutable value and the accepting normalized schema. A replacement value requires another validation.

mechanism

INVARIANT

Unknown schema never becomes inferred schema

A custom codec without a schema reports UnknownSchema. Every enclosing schema query propagates that absence.

mechanism

INVARIANT

Record decoding rejects extra and repeated fields

Closed record decoding rejects unknown members and repeated members. Strict text admission rejects repeated keys before native decoding.

mechanism

PRINCIPLE

Keep wire acceptance separate from native projection

A schema may accept an exact value that a requested native type cannot represent. Callers distinguish schema rejection, parse rejection, and native conversion failure.

PRINCIPLE

Keep callback ownership explicit

Caller mappings own their laws and application checks. A retained schema does not certify a callback's inverse, purity, or encoding behavior.

- Ownership and finite support are recorded in ADR-0001. Number policy is recorded in ADR-0002. Byte-only native admission supersedes ADR-0003 in ADR-0014.

2.2 Pending updates

build **Broader schema families need concrete contracts** 2026-09-20

String constraints, broader numeric and collection constraints, arbitrary alternatives, numeric enums, aliases, and recursive references are outside the profile. Their limits, annotation policies, and validation rules are undefined; see Unsupported schema families.

ADR-0001

verify **Keep numerical oracle scope separate from projection** 2026-09-20

The finite schema manifest compares Boolean acceptance for its selected cases. Broader numerical evidence must compare exact schema acceptance separately from native overflow, safe-integer refusal, and intentional Float rounding.

ADR-0002

verify **Retain complete compatibility verification** 2026-10-02

Published source, legacy wire, recursive references, error paths, numeric boundaries, and deliberate acceptance corrections need separate fixtures. Existing examples and frozen comparators do not prove complete 1.x compatibility.

ADR-0007

verify **Check the legacy and error-path claims against code** 2026-10-17

The 1 MiB decode precheck, its configurable bound and the object-index cutoff match the source. The remaining provisions carried over from the retired 1.x guides are not yet checked against code: the Draft-07 label statements, error-path rendering, number-projection limits and the legacy wire envelopes. The migration examples and the schema oracle exercise part of them.

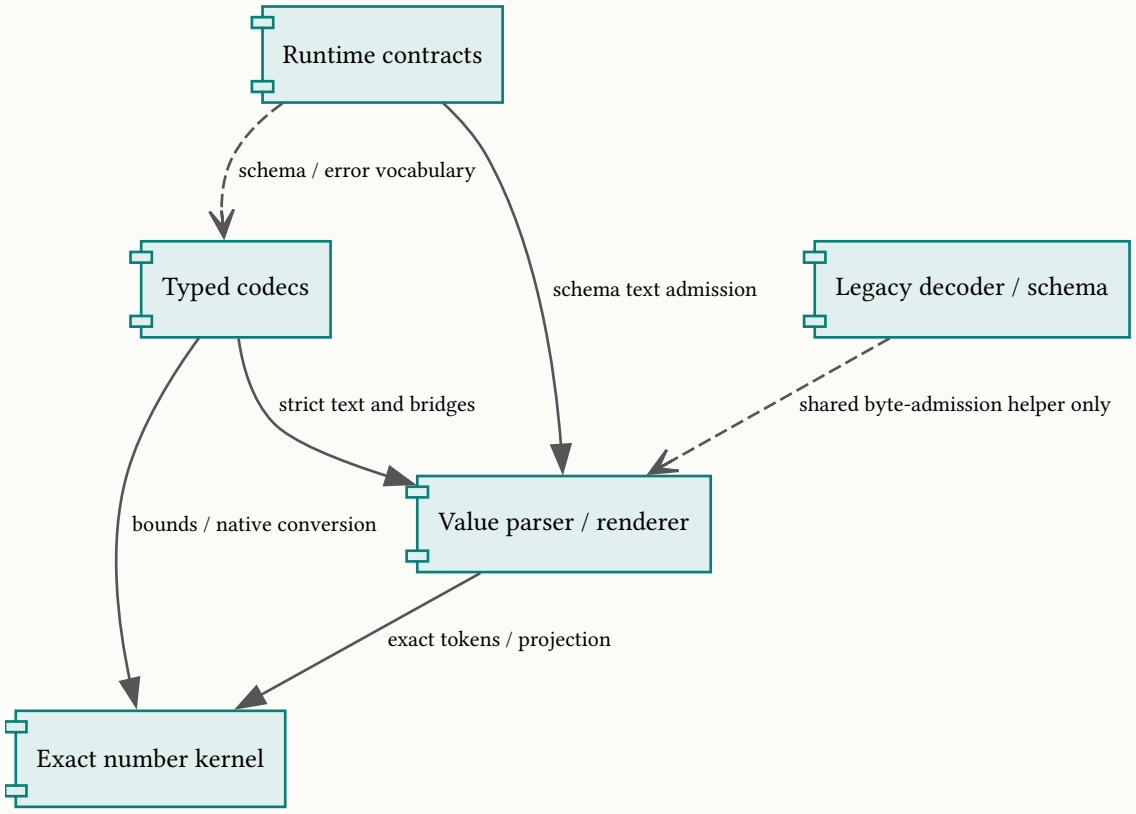
ruling **Consumer evolution policies need explicit contracts** 2026-10-25

Prioritize optional unknown-field handling, dictionaries, basic string/list lengths, one-sided numeric bounds, configurable union envelopes and local recursive references. Before implementation, define accepted input versus emitted output, retention of extra fields, schema projection and reference identity. The current strict defaults and frozen legacy access remain in force; formal deprecation requires a supported migration for recursive data and established wire formats.

2.3 System at a glance

One context owns values, schemas, typed codecs, contracts and legacy compatibility.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Runtime components



LEGEND

component
owned runtime part

call
solid directed link

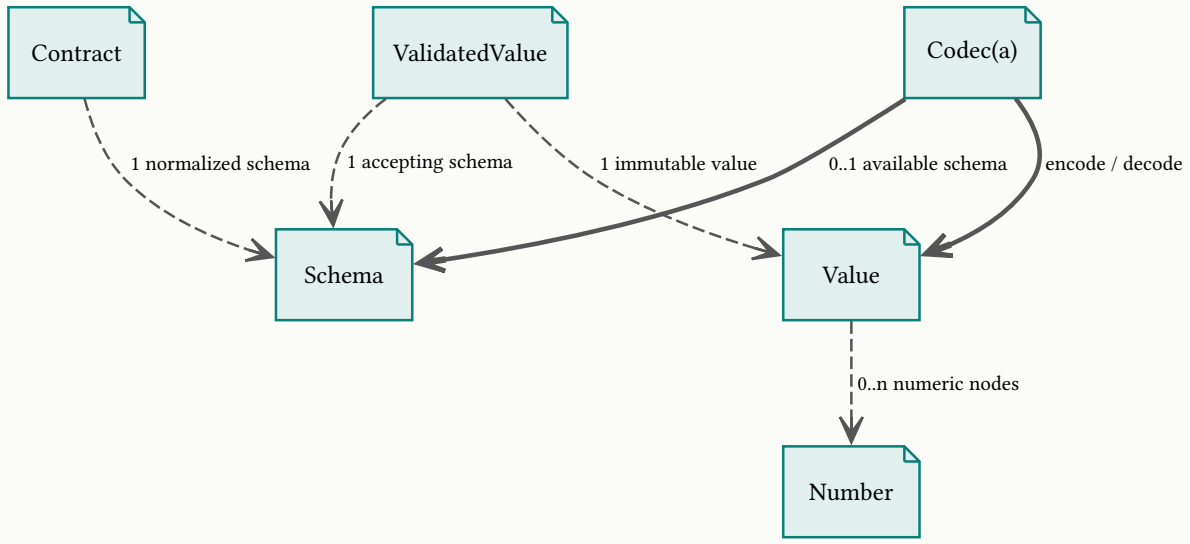
dependency
dashed directed link

Arrows declare call and dependency direction. The legacy surface shares only a byte-admission helper with the strict parser.

- Work is bounded only at the declared admission surfaces. A manually constructed Value, a caller callback, schema discovery, rendering, and validation have no additional timer or allocation quota.

2.4 Whole model

All package-owned domain values are immutable. Schema identity is structural; native types remain caller-owned.



LEGEND



The diagram shows ownership and data relationships. Validation creates another immutable value; it does not mutate the input or register a global identity.

Number

value-object · immutable · JSON contracts · owned by Wire values

Target-independent mathematical numeric value.

ATTRIBUTES

derived **Normalized value** · SmallInteger | Decimal
Small integers have at most 15 decimal digits. Other values carry sign, nonzero coefficient digits without leading or trailing zeros, and a decimal exponent; zero has one positive representation.

RELATIONSHIPS

`1 : 0..n` Number may appear in Values and exact numeric schema bounds.

Value

value-object · immutable · JSON contracts · owned by Wire values

Ordered JSON tree that can retain duplicate-key evidence.

ATTRIBUTES

derived **Kind and content** · Null | Bool | String | Number | Array | Object
Parsing derives the content from admitted text. Public constructors also allow callers to supply content; those constructed values have no parse-admission proof.

RELATIONSHIPS

`1 : 0..n` Arrays own ordered Values. Objects own ordered name/Value pairs; the raw Object constructor permits repeated names.

Schema

value-object · immutable · JSON contracts · owned by Typed codecs

Checked finite structure with optional descriptions.

ATTRIBUTES

derived **Checked structure** · Opaque Tree
Codec discovery or document loading derives structure after checking definition invariants. The public projection never exposes a constructor for unchecked structure.

RELATIONSHIPS

1 : 0..n Property and variant records retain opaque child Schemas with exact name/requiredness and tag/payload associations.

Codec(a)

value-object · immutable · JSON contracts · owned by Typed codecs

Compiled native conversion and discoverable schema capability.

ATTRIBUTES

authored **Native directions** · a -> Result(Value, EncodeError); Value -> Result(a, DecodeError)
Constructors compose directions; custom callbacks are supplied by the caller.

derived **Definition and placeholder** · Deferred schema result; fn() -> a
Definition discovery is computed on demand. This is not a promised cached or constant-time schema lookup.

RELATIONSHIPS

1 : 0..1 A codec supplies one Schema when known. Its type parameter a is supplied by the native program, never by runtime schema text.

Contract

value-object · immutable · JSON contracts · owned by Runtime contracts

Normalized schema retained independently of native callbacks.

ATTRIBUTES

derived **Normalized acceptance** · Finite Tree
Normalization sorts properties, enum labels, and variants without changing associations. Equality for matching strips descriptions.

RELATIONSHIPS

1 : 0..n Validation may produce multiple witnesses. A contract retains no callback, process identity, or native type.

ValidatedValue

value-object · immutable · JSON contracts · owned by Runtime contracts

Immutable acceptance evidence for one retained value.

ATTRIBUTES

derived **Value and accepting schema** · Value × normalized Tree
Only successful validation constructs the public witness. Matching a codec checks normalized schema structure before its native decoder runs.

RELATIONSHIPS

`1 : 1` The witness retains exactly one value and one accepting schema; it cannot bless a modified value.

- Value objects have no persistent identifiers, mutable state fields, commands that change them, or emitted domain events. There is no vacant or revoked state to model: replacement discards the previous value and requires fresh evidence.
- Sum types model Value, SchemaView, typed failures, and union construction. Products model errors with paths, properties, variants, parse locations, and generated source results. Numeric bounds and enum membership are refinements on accepted values.
- Limits are immutable caller-authored settings. The byte setter retains its supplied value; a nonpositive byte limit rejects every input.

2.5 Wire values

This unit owns exact numbers, strict document admission, rendering, and explicit standard-JSON conversion.

Wire value boundary

responsibility

Preserve exact JSON meaning while admitting complete documents within explicit resource limits.

interface

number construction/projection; value.object, parse, parse_bits, to_string, to_json, decoder.

interactions

Codecs and schema loaders use strict admission. Standard-JSON bridges accept another parser's precision and duplicate-key policy.

invariants

No NaN or infinity is a Number. Checked object construction rejects repeated keys without converting the object to a map.

failure

ParseError retains location and reason; numeric construction and projection have separate typed outcomes.

2.5.1 Exact numbers

- Number parsing accepts the JSON numeric grammar: optional minus, integer digits with no leading zero except zero itself, optional nonempty fraction, and optional signed nonempty exponent. Whitespace and a leading plus are invalid.
- Normalization removes coefficient zeros and adjusts the decimal exponent without expanding the value. Comparison first distinguishes signs, then decimal magnitude and significant digits. Mathematical integrality follows normalized coefficient/exponent, so 1.0e2 is an integer.

Operation	Meaning	Failure or target rule
parse(token, limits)	Exact normalized decimal	InvalidSyntax, TokenTooLong, TooManySignificandDigits, ExponentOutOfRange
from_int / to_int	Checked exact native integer conversion	Erlang bignums; JavaScript safe integers only; projection has an output-digit bound
from_float / to_float	Shortest roundtrip decimal / nearest binary64	Nonfinite input or overflow rejects; tiny nonzero numbers may round to zero
from_float_exact / to_float_exact	Exact binary rational / exact mathematical binary64 equality	Overflow, underflow, inexactness, or invalid candidate remain distinct

- Default Number limits are 1,024 token bytes, 800 significant digits as written, and normalized exponent magnitude 1,200. They admit finite binary64 decimal expansions, including subnormals. Token and digit limits below one reject every token; a negative exponent bound rejects nonzero values.
- Native Int construction cannot recover precision already lost before the JavaScript boundary. The unsafe value is rejected even when a host has rounded it to another integer.

BEHAVIOR

Exact decimal survives comparison

Given The numbers 1.2300e2 and 123 were admitted within number limits.

When The application compares them.

Then The result is equality without binary rounding.

level: boundary area: Exact numbers

- The decimal 0.1 can project through the rounded conversion. Exact projection rejects it as FloatInexact; 0.5 projects exactly. Sources: number implementation and target and numeric cases.

2.5.2 Strict document admission

- Text parsing checks UTF-8 byte size before allocating the document tree. On JavaScript, a raw unpaired UTF-16 surrogate then fails with InvalidUnicodeScalar before UTF-8 conversion can replace it. Its location is the end of the valid prefix, measured in UTF-8 bytes and grapheme columns. Byte parsing additionally reports the first invalid UTF-8 byte. A valid document has exactly one root value followed only by JSON whitespace; trailing content rejects.
- Erlang uses OTP JSON hooks to reject duplicate names and retain exact integer values and decimal tokens. Built-in codecs decode the native tree directly; custom Value callbacks convert only their subtree. JavaScript retains the portable exact parser.

Bound	Default	Change
Input bytes	1,048,576	with_max_bytes

- There is no global depth, value-count or numeric-token admission bound, streaming admission or request-wide time budget. The caller has already allocated input text; byte admission is not a heap quota. Consumers own structural policy.
- String admission validates escapes and surrogate pairing. Literal control characters, malformed escapes, and unterminated strings reject. Object duplicate detection compares decoded names, so escaped equivalent names collide.
- Erlang object members follow native map order. JavaScript retains source order. Arrays retain input order. Value conversion copies strings so small retained values do not retain the complete source binary.
- JavaScript ParseError locations retain zero-based byte offsets and one-based line/column. Erlang native syntax and duplicate errors have an unavailable location, represented by zeroes; their description does not invent a source position. Public parse errors retain no input text. Syntax validation completes before typed callbacks run.

BEHAVIOR

The parser refuses repeated names

Given A document repeats the same decoded member name in one object.

When The application parses the document.

Then Parsing fails with DuplicateObjectKey and returns no parsed tree.

level: boundary area: Strict admission

BEHAVIOR

Host text cannot silently replace malformed Unicode

Given JavaScript input text contains a raw unpaired surrogate and is within its byte bound.

When The application parses the text through the strict entry point.

Then Parsing fails with InvalidUnicodeScalar at the first unpaired surrogate without retaining source text in the error.

level: boundary area: Strict admission

- ADR-0014 records byte-only native admission and supersedes the independent limits in ADR-0003. Source: value parser. admission tests and grammar cases cover the boundary. Benchmark observations are not hard heap quotas.

2.5.3 Rendering and standard JSON bridges

- to_string emits compact JSON, escapes strings, and preserves ordered members. It does not normalize object order or reject duplicate members supplied through raw Object construction. Rendering has no independent byte bound.

- value.to_json first tries an exact native integer within 400 output digits, then a finite Float whose shortest roundtrip decimal compares equal to the Number. It returns the offending Number if no exact rendered standard-JSON form exists. This bridge criterion differs from exact binary64 equality: decimal 0.1 has an exact printed JSON form despite an inexact binary representation.
- codec.to_json retains the offending numeric node's Field/Index path as UnrepresentableNumber. Applications propagate that Result before entering APIs that require an infallible JSON encoder; replacing failure with null changes application data.
- value.decoder and codec.decoder consume a native parser's output. That parser owns size, duplicate names, member order, and already-rounded numeric precision. These bridges cannot restore discarded evidence or enforce strict text limits retroactively.
- Sources: rendering and bridge implementation and bridge cases.

2.6 Typed codecs

This unit composes bidirectional conversion for caller-owned native types and discovers the supported schema.

Codec boundary

responsibility

Encode and decode native a using one retained definition and known wire shape.

interface

Primitive codecs, collection codecs, record and union builders, mapping, custom, encode/decode, schema, check, placeholder.

interactions

Strict text parsing produces Value first. Schema publication exposes an opaque schema for consumer admission.

invariants

Built-in constructors preserve their declared wire shape. Unknown child schema propagates through every enclosing schema query.

failure

Value failures return path and Reason. Source definition mistakes panic at first affected use; check returns DefinitionError instead.

2.6.1 Constructor acceptance

Constructor	Native and JSON shape	Acceptance and encoding rule
string / bool	String / Bool	Exact JSON type; placeholder empty string / false
int	Native Int / mathematical integer	Both directions require at most 24 digits and target safety; a longer encoded integer returns ExpectedInt, an unsafe JavaScript integer returns UnsafeInteger
float	Float / number	Decode nearest binary64, refuse overflow; encode shortest roundtrip decimal
number	Number / number	Exact passthrough without native projection
integer_between	Int / bounded mathematical integer	Inclusive safe-native bounds; projection digits derive from bounds
number_between	Number / bounded number	Inclusive exact comparisons
string_enum	Caller value / bare string label	Nonempty unique labels and unique native values; exact case-sensitive labels
pair	`#(a, b)` / two-element array	Exact length; failures retain index 0 or 1
list	List(a) / array	Decode and encode each item with its zero-based index
nullable	Option(a) / null or inner	None is null; Some cannot encode through inner as null
value	Value / any	Identity conversion with AnySchema; text admission still applies

Constructor	Native and JSON shape	Acceptance and encoding rule
success	r / empty object	Decode the retained r from a closed empty object; encoder ignores input

- A codec's accepted native domain can be narrower than its schema. Native Int digit limits and Float overflow are projections, not mathematical schema constraints. try_map may impose application checks that its preserved schema cannot express.
- For caller-authored mappings, a successful encode/decode roundtrip is a caller law. The type system proves neither inverse mappings nor equality to a schema language. A constant success codec has the retained constant as its valid native domain.
- DefinitionError distinguishes duplicate names/tags, empty union/enum, duplicate enum native values, reversed bounds, unsafe JavaScript integer bounds, and a field continuation that does not finish a record. Diagnostics name definition data rather than untrusted input.

BEHAVIOR

A nullable inner null cannot erase Some

Given A native Some value has an inner encoder that produces null.

When The application encodes it through nullable.

Then Encoding returns NullInsideNullable instead of losing its native distinction.

level: boundary area: Constructor acceptance

BEHAVIOR

Integer encoding preserves its decoding domain

Given A native integer exceeds the plain integer codec's digit domain.

When The application encodes it through that codec.

Then Encoding refuses it rather than producing a value that the same codec cannot decode.

Then Explicit integer ranges retain their separate bound-derived digit domain.

level: boundary area: Constructor acceptance

- Sources: constructors, codec cases, and public failure handling.

2.6.2 Record construction and dependent fields

- The **Record builder** binds each required field's value or each optional field's Option. success ends the continuation with the caller's record value. Getters are passed last with get:, so the continuation fixes the native record type before record access is checked.
- Encoding reads each getter in declaration order, encodes that field, then builds its continuation using the actual native field value. Decoding finds and decodes each named member, then builds the continuation using the decoded value. This permits a later field's application check to depend on earlier fields.
- Schema discovery follows continuations using each codec's **Placeholder** and None for optional fields. Names and schema constraints must be independent of those placeholder values; a dynamic application check may differ, but the declared JSON shape cannot.
- The source computes continuation properties on demand. Neither schema discovery nor arbitrary-width record processing promises constant time or linear time for all public definitions. Caller callbacks must remain deterministic and cheap where they are used to describe a schema.
- Optional fields omit None and decode absence as None. Required fields reject absence. Explicit null is rejected by a nonnullable inner codec; nesting optional_field with nullable yields None, Some(None), and Some(Some(x)) for absence, null, and a value.
- Record decoding visits declared fields before it checks unused members. A missing or invalid declared field can therefore precede an extra/repeated member failure. Runtime contract validation checks all member names before property values; the two first-error orders need not coincide.

BEHAVIOR

An optional field preserves present null

Given An optional field contains a nullable string codec.

When The application decodes an explicitly null field.

Then The result for that field is Some(None).

Then Re-encoding includes the field with null.

level: boundary area: Record construction

- A total mapped record has no record-builder continuation capability. A field continuation must remain a field/success record codec; terminating it in another shape is NotARecord.
- An Order total that differs from earlier decoded item quantities can return Custom at Field("total") when the check belongs inside that field's codec. Mapping the whole record reports at the record root. ADR-0004 records the builder and definition-error policy.

2.6.3 Tagged union construction

- union joins variant and unit_variant continuations with match. Each payload variant supplies a typed encoder to the final exhaustive native case; unit variants supply a retained tagged value. Duplicate tags and an empty union are definition errors.
- The Tagged union envelope has tag and value for payload variants, and tag alone for unit variants. All other fields reject. A payload variant requires value; a unit variant rejects even an explicitly null value.
- Decoding checks repeated and unknown member names before selecting tag. Missing tag, nonstring tag, unknown tag, missing payload, and invalid payload return distinct reasons at tag or value paths. Nested unions retain nested envelopes rather than implicitly flattening variants.

BEHAVIOR

A unit variant rejects a payload

Given The selected variant declares no payload.

When The application decodes an object containing value.

Then Decoding returns UnknownField at Field("value").

level: boundary area: Tagged unions

- Encoding trusts match to return the encoder for the selected native case. Reusing a Tagged value from another union is not dynamically certified against a registry; the author owns the union mapping laws. Tests exercise N-ary variants and unit shape in public API cases.

2.6.4 Mappings and custom conversion

- map takes total functions in both directions and derives its placeholder from the inner codec. try_map takes fallible functions plus an explicit placeholder; a conversion failure becomes Custom at the current codec path. Both preserve the inner schema without claiming that business checks are schema-expressible.
- custom accepts functions over Value, an Option(Schema), and a placeholder. None reports UnknownSchema; Some retains a checked schema but does not independently validate every callback result. Captured state and side effects remain callback-owner responsibility.
- placeholder returns description data for composing wrappers. It must never be encoded as fallback application data. A callback invoked during description may encounter this value even when application decoding never accepts it.

BEHAVIOR

Unknown custom schema propagates

Given A custom child codec supplies no schema.

When The application requests its enclosing record's schema.

Then The result is UnknownSchema.

Then The package does not infer an accepting document from callbacks.

level: boundary area: Custom conversion

2.6.5 Paths and safe diagnostics

- EncodeError and DecodeError each carry a root-to-leaf List(PathSegment) and structured Reason. Field(name) and Index(index) locate data. Empty paths identify the root; parse failures become InvalidJson at the root with their own byte location.
- Built-in rendered value reasons omit input values and unknown names from diagnostics. The typed path can still contain a name from the input. Custom messages render verbatim; the caller owns redaction and user-facing safety.
- Reason and ParseReason may gain minor-release alternatives. Callers use a fallback branch where exhaustiveness across upgrades matters. Schema errors, document errors, definition errors, parse errors, value errors, and number projections remain separate outcome families.

2.7 Runtime contracts

This unit loads the finite schema profile, validates immutable values, and checks structural agreement before native decoding.

Runtime contract boundary

responsibility

Interpret an explicit supported schema and preserve acceptance evidence.

interface

from_codec, total from_schema, parse, load, schema, same_schema, validate, value, decode, value_codec.

interactions

The strict parser owns text limits. The schema model owns checked structure. Native decoding retains the caller's codec.

invariants

Unsupported constraints never disappear. Matching preserves requiredness, tuple order, bounds, and variant associations.

failure

LoadError separates JSON admission from document rejection. ValidationError carries first failure path and Reason. Native mismatch is ContractMismatch.

2.7.1 Schema discovery and public projection

- codec.schema returns an opaque `Schema` or UnknownSchema after definition discovery. schema_value renders the schema node; schema_document adds the Draft 2020-12 declaration; schema_json renders the complete document as text.
- `Schema view` has fixed alternatives throughout 2.x: StringSchema, StringEnumSchema, IntSchema, IntegerRangeSchema, NumberSchema, NumberRangeSchema, BoolSchema, PairSchema, ListSchema, NullableSchema, ObjectSchema, UnionSchema, AnySchema, and OtherSchema.
- Descriptions are optional annotations read by codec.description; view looks through them. The latest description at one node replaces the earlier one. Child descriptions remain local to their nodes.
- A future minor-release kind reaches OtherSchema with its rendered schema object. A dedicated projection alternative requires a major release. PropertySchema and VariantSchema may gain fields; consumers read them by label. No current finite kind produces OtherSchema. `ADR-0005` records the evolution contract.

2.7.2 Document loading profile

- load requires an object root with exactly the supported Draft 2020-12 URI in `$schema`. It removes that declaration, interprets the remaining supported shape, checks definition invariants, then normalizes. Nested `$schema` declarations reject as NestedDialect.

Shape	Accepted members	Restriction
Any	Empty object, optionally description; nested true	Root still requires an object and `$schema`; false is not accepted
String / boolean	type; strings optionally enum	Enum is nonempty unique string labels
Integer / number	type; optional minimum and maximum	Both bounds or neither; inclusive and ordered; integer bounds project to safe native Int
Closed object	type, properties, required, additionalProperties	All three structure members explicit; additionalProperties must be false
Homogeneous array	type, items	Exactly one item schema

Shape	Accepted members	Restriction
Pair	type, prefixItems, minItems, maxItems	Two prefix schemas and both lengths equal two; no items member
Nullable	anyOf	Exactly one plain null schema plus one supported object schema
Tagged union	type object, oneOf	Nonempty closed variants; tag const string; exactly declared payload presence

- description is permitted as an annotation on interpreted object schemas. Other keywords reject at their member path. The loader does not accept equivalent arbitrary forms, references, open objects, boolean false, standalone null, type arrays, or general anyOf/oneOf.
- Duplicate raw document members reject before interpretation. MalformedDocument reports observed keyword type and shape violations. UnsupportedDocument reports unhandled keywords, omitted profile-required members, and restrictions on otherwise valid schema forms. UnsupportedDialect retains the declared dialect; InvalidDefinition locates contradictions in the codec model such as reversed bounds or duplicate enum labels/tags.
- Object profile restrictions return UnsupportedDocument: omitted additionalProperties reports OpenObject; omitted properties or required and undeclared required names report UnsupportedSchemaForm at the member or name path. Schema-valued additionalProperties also rejects as UnsupportedSchemaForm. Explicit wrong member types and duplicate required names remain malformed. No omitted member is filled in to admit a new shape.
- Document errors describe the first profile rejection, not complete Draft 2020-12 meta-schema validation. UnsupportedDocument does not certify that the rest of the schema is valid. Boolean roots, missing dialect/type, type arrays, one-sided or nonnative integer bounds, nonstring enum values and tags, and noncanonical tuple/nullable/tagged forms remain refused as unsupported. Actual wrong keyword types remain malformed. Arbitrary third-party protocol schemas are outside the supported profile.

BEHAVIOR

Profile restrictions do not claim malformed schema

Given A schema uses an otherwise valid form outside the finite profile.

When The application requests a runtime contract.

Then Loading refuses it as unsupported at the relevant path.

Then No broader schema form is admitted by changing the diagnostic.

level: boundary area: Document loading

BEHAVIOR

An unsupported keyword remains an error

Given A finite-profile schema includes an unhandled keyword.

When The application loads it as a contract.

Then Loading returns UnsupportedDocument at that keyword.

Then No contract with the constraint discarded is returned.

level: boundary area: Document loading

- Sources: loader, document cases, and contract cases.

2.7.3 Normalization and schema matching

- from_schema is total because every public Schema is checked. from_codec can fail with UnknownSchema and can panic on a source definition mistake. No process-local schema identity token is required.
- Normalization recursively sorts object properties by name, enum labels by text, and variants by tag. It retains associations and child order. Descriptions remain available for rendering; matching separately removes them recursively.
- Contract match is exact equality of those normalized shapes. It is not a solver for equivalence, subtyping, satisfiability, or data migration. A rebuilt equal contract can match a witness.

BEHAVIOR

Different bounds cannot reuse a witness

Given A value was validated under integer bounds zero through ten.

When The application decodes the witness with a codec declaring zero through eleven.

Then Decoding returns ContractMismatch before native conversion.

level: boundary area: Contract matching

- Bound equality must hold even when this particular value lies within both ranges. Matching a custom codec's declared schema still does not certify its executable callbacks. ADR-0006 records the witness contract.

2.7.4 Validation and native conversion

- Validation walks the value and returns the first located failure. It checks exact numeric integrality and bounds without native Int/Float projection; it checks exact pair length, list elements, enum labels, closed property requiredness, and tagged payload shape.
- Closed object and union validation reject duplicate or unknown names before validating declared children. Required properties and payloads retain Field paths; collections retain Index paths. AnySchema accepts the supplied value unchanged, including a manually constructed raw Object with duplicate members.
- Successful validation creates the opaque witness. contract.decode obtains the retained codec's schema, compares normalized shapes, then invokes that codec's native decoder. Schema mismatch and unavailable schema both become ContractMismatch.
- A witness for the mathematical integer 1e30 can still fail codec.int native decoding due to its 24-digit projection bound or JavaScript safety. A witness for a finite exact number beyond binary64 can fail codec.float with FloatOutOfRange. Those outcomes do not invalidate the mathematical schema.
- contract.value_codec supplies Codec(Value) with the contract's schema. Encoding and decoding validate against that same finite schema. Successful conversion returns the supplied Value unchanged, retaining object order and exact numbers. A failure retains the validation path and Reason in the operation's EncodeError or DecodeError. Enclosing record and collection codecs prepend their Field and Index paths.
- This adapter validates structure and supported value constraints on each call without projecting exact numbers into native types. Typed and generated codecs retain their own encoding checks; they do not gain a separate schema-validation pass. Schema acceptance does not certify caller callbacks or business policy. ADR-0006 records the scope and migration cost.
- Direct validate/load have no independent depth, width, or time guard. Parsed text has a byte-admission bound, which is not a heap quota; manually assembled values carry no admission proof. Consumers choose bounded construction/admission where their threat model requires it.
- Object validation searches names with string-specific comparisons up to 40 input members and builds a temporary dictionary above that count. A direct check of the first 32 members rejects an early name failure before the dictionary is built. Accepted values, member order, and first-error paths do not depend on the cutoff, and the dictionary is discarded after each validation. ADR-0009 records the trade-off.

BEHAVIOR

The dynamic codec enforces its contract when encoding

Given The application retains a contract-backed Value codec.

When The application encodes a value through that codec.

Then A value accepted by the contract is returned unchanged.

Then A value refused by the contract returns an encoding failure with the first validation path and reason.

level: boundary area: Value encoding

2.8 Legacy compatibility

The frozen 1.x surface retains established one-way decoding, independent encoders, broader schema description, and recursive references.

Legacy boundary

responsibility

Keep published combinators and established data readable without inferring modern codec laws.

interface

json/blueprint Decoder and FieldDecoder, decode0..decode9, field/optional_field, list, tuples, map, union/enum helpers, self_decoder, reuse_decoder; json/blueprint/schema.

interactions

gleam/json parses after a byte precheck. Legacy schema rendering is independent from native decoder behavior.

invariants

Legacy wire forms remain available explicitly. Opaque Decoder retains its decoder callback, schema description, and definitions.

failure

json.DecodeError and dynamic decode errors retain legacy semantics; no strict finite-contract acceptance is inferred.

Legacy Decoder and FieldDecoder

value-object · immutable · JSON contracts · owned by Legacy compatibility

Immutable one-way native decoder with an independent legacy schema description.

ATTRIBUTES

derived **Callback and description** · Dynamic decoder × SchemaDefinition × definitions
Combinators assemble conversion and description. FieldDecoder additionally retains its property name. No modern schema-validation witness is inferred.

RELATIONSHIPS

1 : 0..n Reusable decoders retain named definitions for references; lazy self-decoding resolves through the caller's retained decoder function.

- Legacy unions use type/data; legacy enum encoders use an enum object. Optional fields treat absence and null alike. Nonempty record decoding ignores unknown fields; decode0 permits scalar/nonempty input.
- decode uses a 1 MiB precheck before gleam/json. decode_with_max_bytes changes that bound. Native parsing owns duplicates and precision; there is no depth/value/token bound on this path.
- self_decoder accepts a lazy decoder and describes a root reference. reuse_decoder hashes the rendered schema definition with SHA-1 to name a `$defs` entry and rewrites root self-references to that entry. Nested recursion needs this explicit rebasing; the hash is a deterministic local description key, not a security authority or schema-equivalence proof.
- The legacy schema description supports type combinations, enum/const, nullable/optional, numeric constraints, string lengths/patterns/formats, array and object constraint families, allOf/anyOf/oneOf/not, references, and boolean schemas. This is a rendering surface, not the modern finite validator or proof that each described constraint is enforced by its decoder.
- The legacy renderer declares Draft-07 while some descriptions use later forms such as prefixItems and `$defs`. Consumers must not infer dialect correctness or general validation parity from that label. Changing only the label cannot prove compatibility.
- Callers that use only the published combinators compile unchanged. Direct construction of Decoder and FieldDecoder records and imports from the internal dynamic module are unsupported: callers build decoders with the combinators and import the standard dynamic decode module. get_dynamic_decoder returns a function from a dynamic value to the decoded value or a list of decode errors with the same expected, found and path fields. ADR-0007 records the compatibility boundary.
- The 2.0 migration guide gives application steps for retaining this API and for separately adopting codecs. Sources: legacy decoder, legacy schema renderer, compiled legacy examples, and the compiled migration example.

2.9 Unsupported schema families

The finite profile is the whole supported schema language. A document outside it is refused and never partially interpreted.

Family	Status	Contract required before support
String constraints	Refused	Length, patterns, and explicit format semantics: Unicode length policy, regex dialect and limits, annotation versus assertion

Family	Status	Contract required before support
Numeric constraints	Only inclusive bounds are supported	Exclusivity and multipleOf: exact arithmetic, projection distinction, hostile-resource budgets
Collections and objects	Closed objects, homogeneous lists and pairs only	Additional item and property constraints and broader object models: uniqueness, contains, pattern properties, limits and error selection
Enums and alternatives	String enums, nullable values and tagged unions only	Numeric enums, aliases, and general alternatives: canonical encoding, overlap, branch failures, equivalence limits
References and recursion	Refused by the runtime profile	Recursive codec and schema families and named definitions: resolution scope, cycles, finite traversal and expansion, recursive laws
Raw unmodeled schemas	Refused	Data for consumer policy without a validation claim: supported interpretation and admission stay explicit
Evolution	No fixtures	Schema and wire migration fixtures: compatibility direction and accepted transformations

- A family enters the profile only when it defines encoding and decoding agreement, schema semantics, rejection behavior, finite resource policy, and independent oracle fixtures. No constraint is silently discarded to accept a remote schema.
- Provider and protocol consumers own admission of a supported schema. They may reject AnySchema, tuple forms, optionality, unions, numeric precision, or future OtherSchema independently of generic local validity. Successful local loading is not proof of remote acceptance.
- The legacy surface describes the broader schema language, but that description is not modern codec or runtime validation support. ADR-0001 records the rationale and scope.

2.10 Verification and host boundaries

Verification distinguishes finite acceptance, native projection and legacy behavior.

2.10.1 Oracle and behavioral evidence

- The independent oracle compares emitted and normalized schemas, codec decoding, and runtime validation with Python jsonschema Draft202012Validator against a frozen manifest. Selected cases carry stable identifiers and expected schemas/instances; missing, extra, duplicate, or replaced cases fail closed.
- The frozen manifest holds the finite cases in named families. Duplicate-key evidence is checked before native export can erase it. Exact-number kernel tests and target float-bit tests remain separate from this Boolean finite corpus. The retained 62-case Erlang arithmetic oracle executes the public number API and independently recomputes its results with Python Decimal and Fraction; both-target unit tests retain native-integer differences.
- Constructor law tests cover admitted native encode/decode and valid re-encoding where applicable. The contract-backed Value adapter enforces finite schema acceptance in both directions. Custom mapping and native projection constraints require their own explicit law scope.
- Hostile tests exercise parser grammar, resource bounds, duplicate keys, malformed/unsupported schemas, and deterministic generated inputs. Wide collection traversal uses tail-recursive paths where the implementation supplies them; caller-constructed unbounded trees retain no guaranteed stack bound.
- The pinned JSONTestSuite byte corpus runs on both targets without network access. Two upstream-valid duplicate-name documents are explicit strict-policy refusals; each implementation-defined fixture has a declared result. Deterministic generated documents and mutations compare acceptance and exact rendered meaning with Python JSON decoding using Decimal. Successful byte admission also agrees with text admission and parse/render/parse. This bounded corpus is not exhaustive fuzzing or general conformance certification.
- Sources: independent checker, frozen manifest, runtime cases, and hostile cases. The oracle document pins the references, states reproducible commands and evidence limits; ADR-0002 records the source attribution.

2.10.2 Native host boundaries

- Erlang/OTP and Node targets exercise the common public contracts with explicit safe-integer, object-order, parse-location and runtime-capacity differences. Number FFI owns host byte handling, integer admission, binary64 parts, exact decimal/

binary comparison, and exact decimal expansion. JSON FFI owns bounded byte checks, JavaScript host-string scalar validation and copied strings; internal dynamic FFI preserves legacy/native decoder conversion.

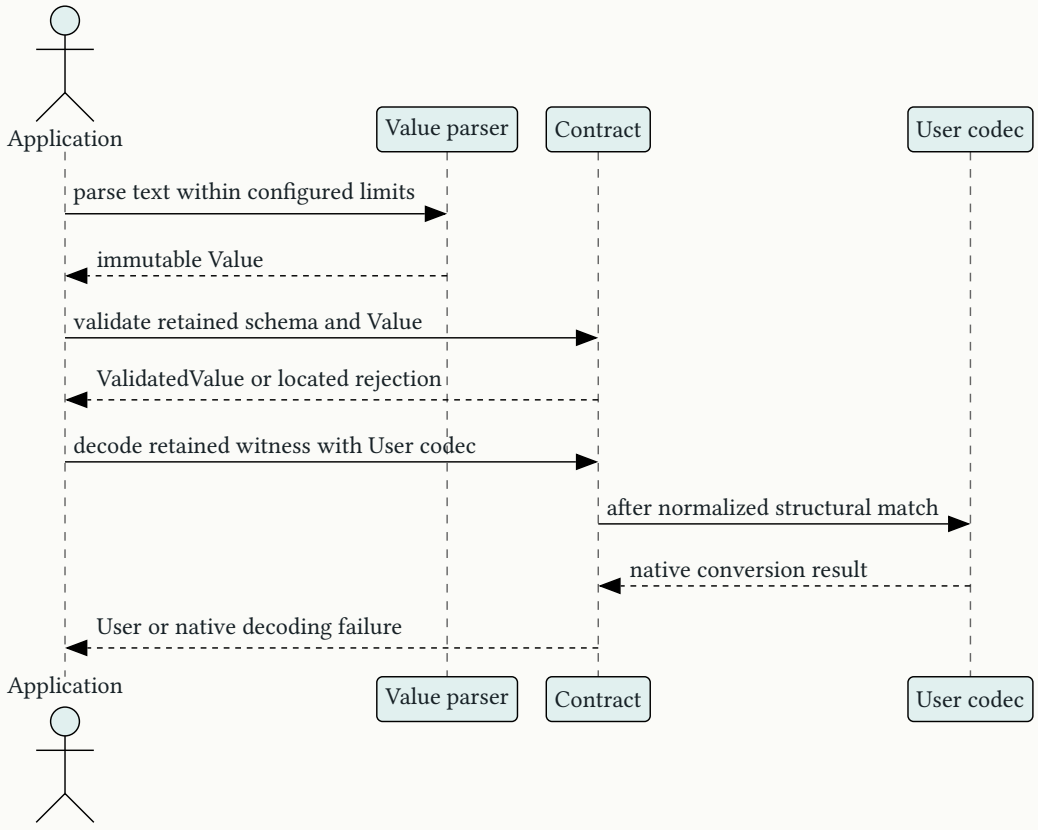
- FFI cannot trust an external host caller's Gleam type annotation. JavaScript adversarial tests inject nonfinite, fractional, unsafe, and previously rounded Int values. Native function name/arity changes need explicit upgrade audits because foreign callers bypass Gleam compiler checks.
- Sources: Erlang number FFI, JavaScript number FFI, Erlang JSON FFI, and JavaScript JSON FFI.

2.11 End-to-end walkthrough

A caller retains one User codec, admits text, publishes its schema, and can validate runtime data before native conversion.

- User has name:String, age:Int bounded zero through 150, optional email:String, and Role encoded as admin/member. The record builder ends in User(name:, age:, email:, role:) and supplies native getters for each field.
- The input {"name":"Ada","age":36,"role":"admin"} passes byte admission, retains exact age, and decodes email as None. Encoding omits email. Schema publication includes closed properties, required name/age/role, exact age bounds, and the enum labels.

SEQUENCE Validation precedes native conversion



The sequence is synchronous. Callback behavior belongs to the retained native codec; no process or transport is implied.

- An age of 151 fails at Field("age") with IntegerOutsideRange. An explicit email:null fails unless that field's inner codec is nullable. A repeated age key fails text admission before conversion.
- A remote schema can be parsed only if it belongs to the finite profile. A default-open object is refused as an unsupported form. A consumer needing broader remote schemas must preserve the unsupported result rather than substitute a permissive contract.
- Mapping User into an application-specific type retains the wire schema. The application handles Custom failures explicitly and owns mapping laws. Admitting a tool or provider request remains that consumer's separate decision.

CONTEXT

Source generation

3 Source generation

3.1 Foundation

Development tools return reviewable Gleam source for native types and codecs. The application build owns writing, formatting and compiling that source.

GOAL

Generate reviewable native source

A caller-authored typed definition or an admitted closed-object schema produces Gleam source that compiles to a codec with the behavior of the public combinators.

INVARIANT

Generation never discards a supported constraint

Descriptions, exact bounds, requiredness, and wire names survive lowering. A shape the generator cannot lower returns a located failure instead of a weaker codec.

partial

INVARIANT

A failed generation returns no module

Invalid names, name collisions, and unsupported shapes return a typed failure. No partial source is returned, and generation performs no file effect.

mechanism

PRINCIPLE

Keep generation out of application runtime

Generated applications depend on the contracts runtime and their compiled source. The build compiler, not schema text at runtime, supplies each native type.

- The generators share the json_blueprint package and release version; their placement is recorded in ADR-0008. Contracts supplies the codec, schema, and runtime helpers that generated source uses, as declared in the context map.

3.2 Pending updates

build **Broader runtime-schema generation is not designed** 2026-10-08

Selected naming overrides, shared named definitions, string and numeric enums, tagged and broader unions, aliases, recursive forms, and a general command-line workflow are outside generation. The build compiler creates each native type, so parsing a schema at application runtime cannot supply a typed codec. Formatter integration and upgrade fixtures remain tooling work beyond the focused consumer integration tests; committed-output freshness is reviewed manually.

ADR-0008

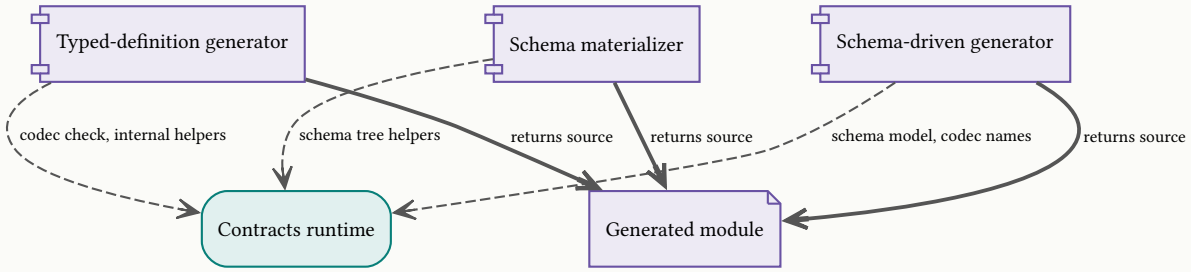
verify **Check the generated-record claims against code** 2026-10-17

The optional, nullable and absent three-state rule, the exact Number against native Int projection and the naming rules for nested stems were carried over from the retired consumer guide. The compiled consumer exercises them, but no check compares each design statement with the generator source.

3.3 System at a glance

Three generators lower definitions and schemas to source; each returns a module and starts no process.

VIEWPOINT DATAFLOW · ALTITUDE L3 · COMPONENTS Generators and output



LEGEND

component
owned runtime part

value object
attribute-defined value

bounded context
domain owner

dependency
dashed directed link

dataflow
weighted directed link

Generators run at build time. Generated source calls the public codec combinators of the contracts runtime.

3.4 Source generation

Development tools return reviewable source. Runtime document loading never constructs a new native type.

3.4.1 Typed-definition generator

Codegen definition boundary

responsibility

Lower a typed definition to a standalone Gleam module with matching runtime behavior.

interface

json/blueprint/codegen: Definition, named_mapping, enum_variant, runtime, compile.

interactions

The runtime codec is checked before lowering. Generated source uses runtime internal/generated helpers under the versioned compatibility contract.

invariants

Names/references are explicit. Runtime conversion does not invoke generation.

failure

CompileError preserves invalid names/references/imports, invalid definitions, unavailable schemas, unsupported constructors, and native-number restrictions.

Definition(a) value-object immutable JSON contracts owned by Source generation

Immutable source-lowering definition whose native type belongs to the compiled caller.

ATTRIBUTES

derived **Runtime and source model** · Codec(a) × native type expression × lowering function
Definition constructors compose a runtime codec and source instructions. Source names remain explicit input where functions cannot be inspected.

RELATIONSHIPS

1 : 1 Definition retains exactly one runtime codec. Its properties and mappings may retain multiple child definitions.

Build value	Owned data	Invariant and failure
Properties(r, a)	Record builder, type expression, names, property lowering	Duplicate names fail definition checking; object closes the accumulated fields
Mapping(a, b)	Two runtime functions and two source references	Source forms validate; callback/reference behavioral equality remains caller-owned
EnumVariant(a)	Wire label, native value, source constructor reference	Reference syntax checks before admission; enum definition checks label/value uniqueness
GeneratedModule	Relative path, content, optional codec fingerprint	No file is written automatically; compile and materialization return distinct result records

- Codegen definition carries a Codec(a), a native type expression, and lowering instructions. Properties combines typed required/optional fields for a closed object. Mapping joins runtime functions with explicit qualified references because Gleam cannot recover source names from closures.
- Supported definitions are string, int, number, bool, integer ranges, pair, list, nullable, finite string enum, required/optional properties, object, imap with named_mapping, and descriptions. A definition can exist at runtime while its native generated path is unsupported; compile refuses exact-number forms lacking supported native lowering with NativeNumberUnsupported.
- compile validates module path and accessor name, parses native type references, calls codec.check, lowers the definition, checks native support and import agreement, then returns a Generated module. The result contains relative .gleam path, content, and SHA-1 content fingerprint.
- Generated schema helpers trust generator-checked trees. Direct caller construction through internal tree helpers bypasses ordinary schema admission and owns the resulting invariants.
- Qualified callback/constructor names and type/import references are validated as source forms. Compilation checks referenced function types; same-typed callbacks paired with different source references remain a caller trust boundary. Parity tests must compare interpreted and generated behavior.
- A generated module exports Value encode/decode, strict text encode/decode, the explicitly distinct native JSON decoder, a schema accessor, and a codec accessor. Strict generated decoding uses the common Value parser; native decoding uses gleam/json after a 1 MiB byte check and does not inherit strict depth/number policies.
- Generation returns source only. The caller owns file writing, formatting, compiling, reviewing, committing generated output, and drift checks. Package internal/generated helper names/signatures consumed by checked-in output remain stable within a major version; internal marking alone does not eliminate that compatibility obligation.
- Sources: generator, runtime helper contract, compiled source fixtures, and build workflow. ADR-0008 records ownership.

3.4.2 Schema materialization

- The internal materializer exports functions that implement Schema materialization: they reconstruct checked schema structure without constructing codec callbacks. It validates module/accessor names, rejects duplicate accessor names, sorts exports by accessor name, and escapes labels as data.
- Materialized schema functions may allocate a schema tree on each call. They generate neither native record types nor native codecs, and they establish no constant-evaluation or constant-time lookup guarantee.
- Lowering emits supported tree constructors and preserves descriptions, child schemas, property requiredness, and variant payload presence. Unknown or unsupported structure returns a located MaterializationError rather than dropping constraints.
- Sources: materializer and schema materialization evidence.

3.4.3 Schema-driven native generator

Schema generation boundary

responsibility

Generate native record types and a typed codec from an admitted closed-object schema.

interface

json/blueprint/codegen/schema.compile(module_path: String, name: String, schema: codec.Schema) returns Result(codegen.GeneratedModule, schema.CompileError).

interactions

The build compiler creates native types from returned source. Generated codec expressions use public codec combinators.

invariants

No supported constraint is discarded. Native fields retain their original wire names through their codecs.

failure

Typed generation failures distinguish invalid names, conflicting names, and unsupported shapes with logical schema paths. No partial module is returned.

- The Schema-driven generator accepts only a closed-object root. The explicit name is a validated lowercase accessor stem: order selects public Order and `order_codec()`. The module path follows the existing generator's source-path validation.
- Generated root and nested record types are public so consumers can construct native values. Only the root codec accessor is public; nested codec functions are private. Generated applications depend on JSON Blueprint and their compiled source; generation runs only when explicitly invoked.
- The generated module uses ordinary public codec constructors and record continuations. It does not emit a second Value validator or route through the typed-definition generator's specialized emitter. Caller mappings from generated records to business types remain application-owned.
- Calling the root accessor constructs its codec and prebinds child codec definitions before field continuations. Exact bounds are reconstructed during that construction; retaining the returned codec reuses those child definitions and bounds across conversions. Each accessor call constructs a new codec without a global cache.

Admitted schema shape	Native type	Preserved behavior
Closed object	Public named record	Requiredness, original wire names, and closed-object decoding
String / boolean	String / Bool	Primitive conversion
Integer / inclusive integer range	Int	Target-specific native projection and inclusive bounds
Number / inclusive number range	Number	Exact mathematical values and exact inclusive bounds
List / pair	List(a) / `#(a, b)`	Item conversion and exact pair length
Nullable	Option(a)	Existing nullable codec semantics
Optional object property	Option(a)	None omits the property; Some encodes through its child codec

- Optional nullable properties use Option(Option(a)). None means missing, Some(None) means explicit null, and Some(Some(value)) means present non-null data. Repeated nullable wrappers retain the existing codec's encoding rules.
- Schema descriptions and exact numeric bounds survive generation. Object properties are sorted by Unicode codepoint order of their original wire names before traversal and source emission, so input property order does not select names, layout, or the fingerprint.
- Native Int bounds retain the ordinary codec's target-specific limits. A bound admitted on Erlang may exceed JavaScript's safe-integer range; generated source must be checked on each intended target, where warnings-as-errors compilation and codec definition validation refuse unsafe integer bounds. Exact Number bounds remain independent of native Int and Float representation.
- String enums, tagged unions, AnySchema, future OtherSchema, and every unsupported root produce located generation failures. Admitting a schema for runtime validation does not imply admitting it for native generation.

3.4.4 Generated names and failures

- Native field names retain lowercase ASCII letters and digits. Each uppercase ASCII letter becomes lowercase and starts an underscore-separated word; runs of non-ASCII or nonalphanumeric delimiters collapse to one underscore. Leading and trailing underscores are removed, digit-leading results receive `field_`, Gleam keywords receive a trailing underscore, and an empty result is rejected.
- A nested object stem joins its parent stem and normalized field name with an underscore. List children append `_item`; pair children append `_first` or `_second`; nullable wrappers keep the stem. Pascal-case conversion supplies record type and constructor names.
- Native field collisions are rejected within one record. Generated type collisions are rejected across the module, including conflicts with builtin or used imported type names. Failures retain both origins as schema locations or reserved-name markers; the generator never assigns ordinal suffixes to hide a collision.
- CompileError distinguishes invalid module paths, invalid root or field names, name collisions, and unsupported shapes. Locations use codec.PathSegment in the logical canonical rendered schema, including properties and collection positions; they are not byte offsets in the original document.

- Module source and its content fingerprint are returned as a Generated module. The caller owns writing, formatting, compiling, reviewing, and drift checking the source. Generation starts no process, stores no global state, and adds no runtime cache.

BEHAVIOR

A supported schema supplies compilable native records

Given An admitted closed-object schema uses supported child forms and nonconflicting names.

Given Its integer bounds fit the intended target's native Int.

When The caller generates the supported schema module.

Then The returned source compiles into public root and nested native records with a public root codec accessor.

Then The codec preserves the schema's supported constraints, descriptions, and wire names.

level: boundary area: Schema generation

BEHAVIOR

Native outbound values retain numeric constraints

Given A generated native record contains a value outside a declared inclusive numeric bound.

When The application encodes the record through its generated codec.

Then Encoding fails with the ordinary codec's located bound failure.

level: boundary area: Schema generation

BEHAVIOR

Optional nullable fields preserve three states

Given A generated record has an optional nullable property.

When The application converts a value through the generated codec.

Then Missing, explicit null, and present non-null values remain distinct native and wire states.

level: boundary area: Schema generation

BEHAVIOR

Unrepresentable source fails before a module is returned

Given A schema has an unsupported shape or an invalid or conflicting generated name.

When The caller generates its module.

Then Generation returns the typed failure and its applicable logical schema locations.

Then No partial source module is returned and no constraint is silently removed.

level: boundary area: Schema generation

BEHAVIOR

Equivalent property order regenerates identical source

Given Two supported schemas differ only in object property order and use the same selected module path and root name.

When The caller generates both modules.

Then Their source and content fingerprints are identical.

level: boundary area: Schema generation

- The separate consumer loads an order schema, compiles generated records, adds an application-owned envelope and dispatch policy, and tests failure recovery. Its separate generation task owns file effects and works before the generated application

module exists. The consumer check regenerates source into a disposable consumer, runs the consumer tests on both targets, and compiles a production copy without development dependencies.

- The consumer's two-target corpus uses portable native bounds. Acceptance comparison distinguishes exact wire acceptance from native Int projection failures; it does not claim full-profile parity or general JSON Schema conformance. CPU, construction, generation, and allocation measurements are separate evidence and impose no timing assertion on semantic tests. ADR-0008 records the implementation choice.

3.4.5 Generated parity and verification

- Generated fixture tests format and compare regenerated source, compile retained modules, compare schema bytes and behavior, and run both targets. Named mapping references need parity evidence beyond source type checking.
- A separate consumer regenerates its module, compares it with committed output, runs its tests on both targets, and builds an isolated production copy without development dependencies. The same consumer owns the acceptance corpus and the comparison harness for cost measurements.
- Benchmarks of generation, codec construction, and conversion are observations rather than semantic limits. ADR-0010 records the method and the accepted findings.

Glossary

Every term the layer declares, with the context that owns it.

Admission limits

owned by contracts

The caller-authored byte bound checked before text parsing. Consumers own policies for the decoded structure; explicit standalone number parsing can apply separate numeric limits. A value assembled without parsing carries no admission proof.

Codec

owned by contracts

A typed definition joining native encoding, native decoding, a placeholder, and an available or unavailable schema. The native type parameter belongs to the caller's compiled program.

Codegen definition

owned by generation

A development-time value joining a runtime codec, a native type expression, and source-lowering instructions. Named callbacks include explicit source references.

Consumer

owned by contracts

A separate package or application that publishes, admits, or executes with codecs and schemas and owns its own protocol, provider, and business policy. A consumer imports only public modules of this package.

Contract

owned by contracts

A normalized runtime-validation schema that retains no native type or application callback.

Contract match

owned by contracts

Equality of finite schema structure after normalization and removal of descriptions.

Definition error

owned by contracts

A contradiction in a codec definition, distinct from invalid input or unavailable schema.

Exact number

owned by contracts

A finite mathematical JSON number represented by a normalized decimal coefficient and exponent, with one positive zero. Its value is independent of a target's native integer or binary64 representation.

Finite profile

owned by contracts

The supported schema language of primitives, exact bounds, closed objects, pairs, lists, nullable values, string enums, tagged unions, and any value. It is narrower than JSON Schema Draft 2020-12.

Generated module

owned by generation

A returned Gleam source artifact with its relative module path and, for codec generation, a content fingerprint. Writing and compiling it belong to the caller's build workflow.

JSON value

owned by contracts

An immutable JSON algebra value with ordered object members and an exact number alternative. Duplicate object members remain representable until a checked boundary rejects them.

Legacy decoder

owned by contracts

The frozen 1.x one-way decoder with an independent legacy schema description. Its established data envelopes and acceptance rules differ from Codec.

Native projection

owned by contracts

A checked conversion of an exact number into a target's Int or Float representation. Wire acceptance and successful projection are separate facts.

Oracle corpus

owned by contracts

The frozen manifest of finite-profile cases with stable identifiers, expected schemas, and instances. An independent checker compares package output with a Draft 2020-12 validator, and a missing, extra, repeated, or replaced case fails.

Placeholder

owned by contracts

A codec's native value used to describe a definition without input and to construct a failing native decoder. It need not be valid application data.

Record builder

owned by contracts

A continuation-based codec definition that binds decoded fields by name and ends with a native record constructor. Each field also supplies its native getter.

Schema

owned by contracts

An opaque, checked description of the finite JSON acceptance profile. Descriptions are annotations on that structure.

Schema materialization

owned by generation

Generated functions that reconstruct checked schema structure without constructing codec callbacks. A materialized function may allocate a schema tree on each call.

Schema view

owned by contracts

The stable public projection of one schema node with opaque schema children. Its fallback carries a future kind's rendered JSON Schema object.

Schema-driven generator

owned by generation

The development-time function that turns an admitted closed-object schema into public native record types and a root typed codec accessor. It never constructs a native type from schema text at application runtime.

Tagged union

owned by contracts

A closed object sum selected by a string tag, with a payload exactly when that selected variant declares one.

Unsupported document

owned by contracts

A schema document that is well-formed for its keywords but outside the finite profile. It is refused at the first unsupported feature, and the refusal does not certify the rest of the document.

Avoid: invalid schema — a malformed keyword value is a separate outcome, and an unsupported form is valid JSON Schema.

Validated value

owned by contracts

An opaque immutable pair of a JSON value and the normalized schema that accepted it. Its evidence applies to that value under that schema.